

**Name:** Godwin M. Labaya

**Course & Section:** BSIT – 3 (IT342)

## 1. Github Link

<https://github.com/godwinlabaya/IT342-Labaya-DisasterAidConnect.git>

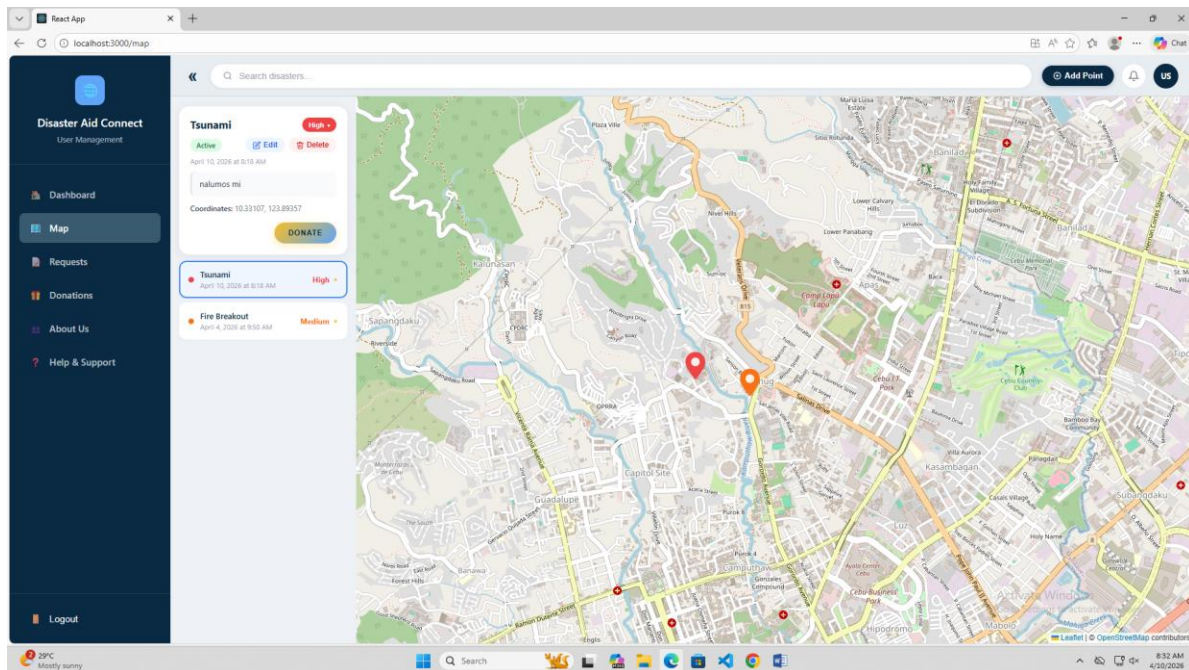
## 2. Final Commit

Commit link: <https://github.com/godwinlabaya/IT342-Labaya-DisasterAidConnect/commit/68e2463618937a7bd74f5b9a42a7a4483baa4042>

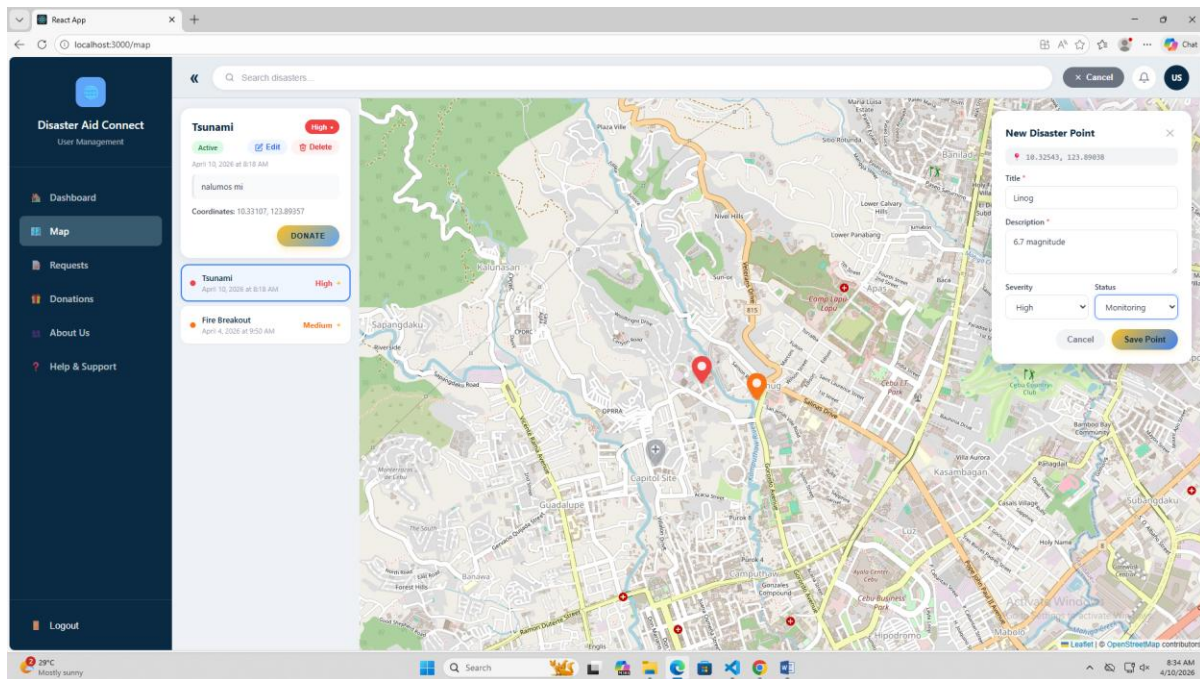
Commit hash: dd9a51d (origin/feature/map-api-implementation, feature/map-api-implementation) IT342 Phase 3 – Web Main Feature Completed

## 3. Screenshots

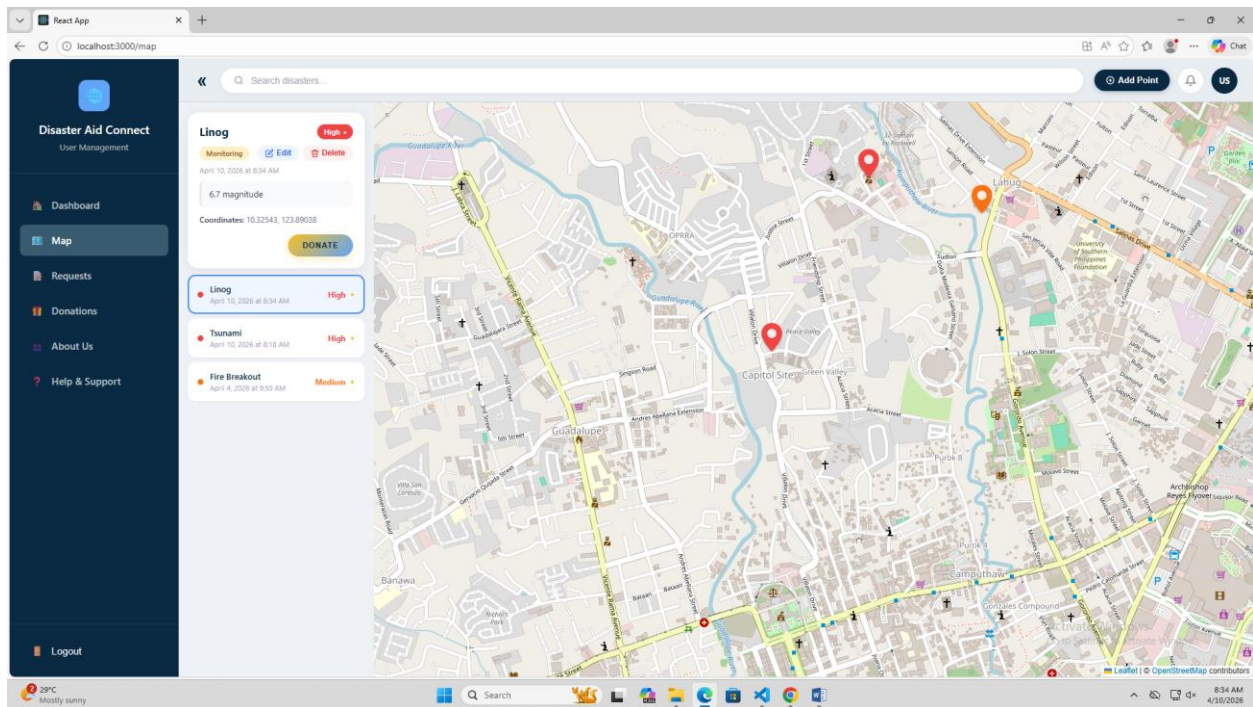
*Main feature page/screen*

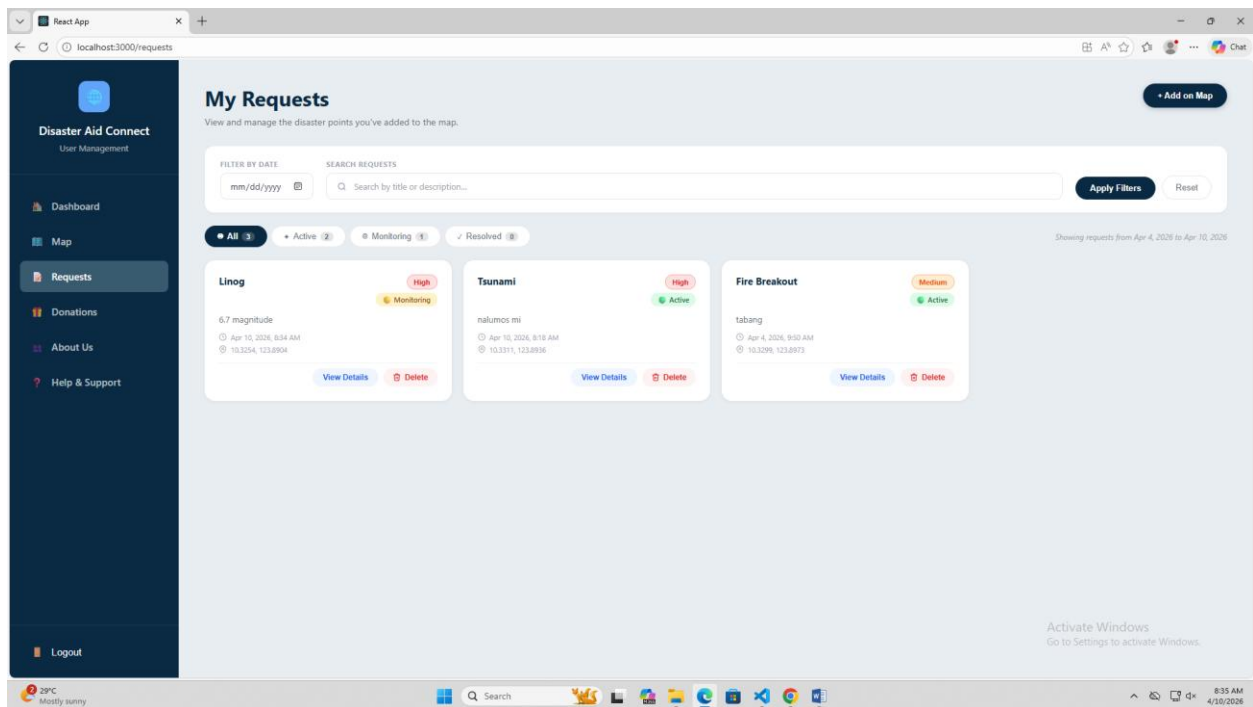


## Adding or using the main feature



## Successful output/result





## Database record related to the feature

The screenshot shows the Supabase Table Editor for the 'disasters' table. The table has the following columns: id, title, description, severity\_level, status, latitude, and longitude. The data is as follows:

| id                                   | title         | description   | severity_level | status     | latitude         | longitude        |
|--------------------------------------|---------------|---------------|----------------|------------|------------------|------------------|
| 09cfd1ea-dd3c-4c57-a447-198a2eacec15 | Tsunami       | nalumos mi    | High           | Active     | 10.3310686967033 | 123.893573284149 |
| b8431d83-b56d-4ed8-b64b-f955d166b777 | Linog         | 6.7 magnitude | High           | Monitoring | 10.3254323298429 | 123.890376091003 |
| f2d37b0f-82b8-4a6e-bcf9-68de752b9381 | Fire Breakout | tabang        | Medium         | Active     | 10.329905585918  | 123.897307282183 |

The bottom of the page shows a Windows taskbar with the date 4/10/2026 and time 8:36 AM.

## 4. Short summary

### Description

The Map Feature is the core interactive component of Disaster Aid Connect. It allows authenticated users to visualize active disaster zones on a live map, report new disaster incidents by dropping a location pin, and manage previously submitted disaster reports.

The map is powered by OpenStreetMap tiles rendered through React-Leaflet and is fully integrated with Supabase, which handles data persistence, real-time updates, and Row-Level Security (RLS) to enforce ownership rules.

### Inputs and validations used

The primary input is a **map click event** captured by React-Leaflet's useMapEvents hook. When the user is in "Add Mode," clicking the map captures the latitude and longitude from the Leaflet LatLng event object and drops a temporary grey pin at that location.

| <i>Field</i>         | <i>Type</i>        | <i>Required</i> | <i>Validation Rules</i>                                              |
|----------------------|--------------------|-----------------|----------------------------------------------------------------------|
| Title                | Text Input         | Yes             | Must not be empty; .trim() applied                                   |
| Description          | Text Area          | Yes             | Must not be empty; .trim() applied                                   |
| Severity Level       | Dropdown           | Yes             | Enum: Low, Medium, High, Critical – enforced by DB CHECK constraint  |
| Status               | Dropdown           | Yes             | Enum: Active, Monitoring, Resolved – enforced by DB CHECK constraint |
| Latitude / Longitude | Auto-filled        | Auto            | Set from map click; not user-editable; stored as double precision    |
| Created By           | Auto-filled (UUID) | Auto            | Populated from supabase.auth.getSession(); not shown in the form     |

### Client-Side (React):

- Empty title → shows inline error: *"Title is required."*
- Empty description → shows inline error: *"Description is required."*
- Error renders in a red box above the Save button
- Save button shows *"Saving..."* and is disabled during the async operation

## Server-Side / Database (Supabase RLS + CHECK constraints):

- severity\_level must match the allowed enum values (DB CHECK constraint)
- status must match the allowed enum values (DB CHECK constraint)
- INSERT only succeeds if auth.role() = 'authenticated'
- UPDATE and DELETE only succeed if auth.uid() = created\_by (RLS enforced)
- Users cannot edit or delete another user's disaster points

## How the feature works

**Page Load:** On mount, the app retrieves the logged-in user's session, fetches all disasters from Supabase ordered by date, and renders them as colored pins on an OpenStreetMap centered on Cebu City. The first disaster is auto-selected and shown in the left panel.

**Adding a Point:** The user clicks "Add Point," then clicks anywhere on the map to drop a temporary pin. A form slides in asking for Title, Description, Severity Level, and Status. On submit, the data is saved to Supabase and the new pin appears on the map immediately.

**Editing a Point:** If the logged-in user is the creator of the selected disaster, Edit and Delete buttons appear. Clicking Edit opens the same form pre-filled with existing values. Only the text fields are editable — coordinates stay fixed. Changes are saved via a Supabase PATCH call.

**Deleting a Point:** Clicking Delete shows a confirmation dialog with the disaster name. Confirming permanently removes the record from Supabase and the pin disappears from the map.

**Search:** Typing in the search bar filters both the left-panel list and the map markers in real time using the same filtered array, keeping them always in sync.

## API endpoints used

All requests go through the **Supabase JavaScript client**, which wraps the Supabase PostgreSQL API. Every request is authenticated via the user's JWT session token.

**Base URL:** `https://<PROJECT_REF>.supabase.co/rest/v1/disaster`

| <i>Operation</i>    | <i>HTTP Method</i> | <i>Supabase Client Call</i>                                                                   |
|---------------------|--------------------|-----------------------------------------------------------------------------------------------|
| Fetch all disasters | GET                | <code>supabase.from('disasters').select('*').order('created_at', { ascending: false })</code> |
| Insert new disaster | POST               | <code>supabase.from('disasters').insert([payload]).select().single()</code>                   |

|                    |        |                                                                                        |
|--------------------|--------|----------------------------------------------------------------------------------------|
| Update disaster    | PATCH  | <code>supabase.from('disasters').update(changes).eq('id', id).select().single()</code> |
| Delete disaster    | DELETE | <code>supabase.from('disasters').delete().eq('id', id)</code>                          |
| Get logged-in user | GET    | <code>supabase.auth.getSession()</code>                                                |

### External Map API

| <i>Property</i> | <i>Detail</i>                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------|
| Provider        | OpenStreetMap (OSM) – free, no API key required                                                                     |
| Tile URL        | <a href="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png">https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png</a> |
| Library         | React-Leaflet v4 + Leaflet.js (via npm install leaflet react-leaflet)                                               |
| Cost            | Completely free and open source                                                                                     |
| Attribution     | © OpenStreetMap contributors (displayed in map corner per OSM terms)                                                |

### Database table/s involved

Primary Table: Disasters

| <i>Column</i>  | <i>Data Type</i> | <i>Constraint</i> | <i>Description</i>                                |
|----------------|------------------|-------------------|---------------------------------------------------|
| id             | uuid             | PRIMARY KEY       | Auto-generated via <code>gen_random_uuid()</code> |
| title          | text             | NOT NULL          | Short name for the disaster incident              |
| description    | text             | NOT NULL          | Detailed description of the situation             |
| Severity_level | text             | NOT NULL, CHECK   | One of: Low, Medium, High, Critical               |
| Status         | text             | NOT NULL, CHECK   | One of: Active, Monitoring, Resolved              |
| Latitude       | double precision | NOT NULL          | Geographic latitude of the disaster pin           |
| Longitude      | double precision | NOT NULL          | Geographic longitude of the disaster pin          |
| Created_by     | uuid             | FK -> users.id    | ID of the user who reported the disaster          |
| Created_at     | timestampz       | DEFAULT now()     | Timestamp when the record was inserted            |

## RLS Policies on disasters

| <i>Policy</i>                            | <i>Operation</i> | <i>Condition</i>              |
|------------------------------------------|------------------|-------------------------------|
| Authenticated users can read disasters   | SELECT           | auth.role() = 'authenticated' |
| Authenticated users can insert disasters | INSERT           | auth.role() = 'authenticated' |
| Creators can update their disasters      | UPDATE           | auth.uid() = created_by       |
| Creators can delete their disasters      | DELETE           | auth.uid() = created_by       |